Ανάπτυξη λογισμικού

Nearest Neighbor Search

ΓΕΩΡΓΙΟΣ ΤΣΕΡΒΕΝΗΣ
ΝΙΚΟΛΑΟΣ ΤΣΟΛΚΑΣ

12-01-2025

Contents

1. Επισκόπηση....3
1.1 Πως δουλέψαμε....3
1.2 Προδιαγραφές Μηχανήματος....3
2. Σύνοψη Εφαρμογής....4
2.1 Libraries....4
Data Vectors....4
Graph....4
Input....4
Utilities....4
Vamana....4
2.2 Executables....5
Main Executables....5
Split Executables....5
2.3 Tests....6
3. Απόδοση – Πειράματα....7
Σταθερές Τιμές Πειραμάτων....7
Τεχνικές Βελτιστοποιήσεις....7
3.1 Αρχική Απόδοση....8
3.2 Βελτιστοποιήσεις....9
3.2.0 Squared Euclidean Distance....9
Περιγραφή....9
Αποτελέσματα....9
Παρατηρήσεις....9
3.2.1 Min Distance....10
Περιγραφή....10
Αποτελέσματα....10
Παρατηρήσεις....10
3.2.2 Πίνακας Αποστάσεων....11
Περιγραφή....11

Αποτελέσματα....11
Παρατηρήσεις....11
3.2.3 Αρχικοποίηση Γράφων με τυχαίες ακμές....13
Περιγραφή....13
Αποτελέσματα....13
Παρατηρήσεις....13
3.2.4 Αρχικοποίηση medoid στον Vamana με τυχαία σημεία....14
Περιγραφή....14
Αποτελέσματα....14
Παρατηρήσεις....14
3.3 Παράλληλη Εκτέλεση....15
3.3.0 OpenMP....15
3.3.1 Υπολογισμός Αποστάσεων....15
Περιγραφή....15
Αποτελέσματα....15
Παρατηρήσεις....15
3.3.2 Υπολογισμός ανεξάρτητων Γράφων σε Stitched Vamana....15
Περιγραφή....15
Αποτελέσματα....15
Παρατηρήσεις....15

1. Επισκόπηση

1.1 Πως δουλέψαμε

Μετά την ανακοίνωση κάθε εκφώνησης, κανονίζαμε μια απομακρυσμένη σύσκεψη κατά την οποία συζητούσαμε πώς θα αντιμετωπίσουμε το αντίστοιχο κομμάτι της εργασίας.

Μοιράζαμε τμήματα των ζητούμενων, όπου μπορούσαμε, και τα υλοποιούσαμε ξεχωριστά χρησιμοποιώντας version control μέσω Git και GitHub. Όταν ερχόταν η στιγμή που έπρεπε να συγχωνεύσουμε τα επιμέρους τμήματα ή όταν έπρεπε να υλοποιήσουμε σημαντικά τμήματα της εργασίας, είτε τμήματα τα οποία δεν μπορούσαμε να μοιράσουμε, δουλεύαμε μαζί σε κοινό μηχάνημα.

1.2 Προδιαγραφές Μηχανήματος

Τα πειράματα που παρουσιάζονται σε παρακάτω ενότητα έγιναν σε μηχάνημα με τις εξής προδιαγραφές:

Λειτουργικό Σύστημα	Ubuntu 24.04 LTS (WSL 2.0)
Τύπος Συστήματος	X64
Επεξεργαστής	Intel i5-8600K CPU @ 3.60GHz, 3600 MHz, 6 Core(s)
RAM	16 GB

2. Σύνοψη Εφαρμογής

Η ανάπτυξη της εργασίας έγινε σε C++ κυρίως με τη χρήση Clion πάνω σε WSL 2.0 και GitHub Actions για τον έλεγχο και την επαλήθευσή της.

2.1 Libraries

Data Vectors

Η βιβλιοθήκη Data Vectors περιέχει μόνο την κλάση DataVector η οποία χρησιμοποιήθηκε για να αναπαραστήσει την οντότητα των δεδομένων - διανυσμάτων και των απαραίτητων συμπεριφορών τους.

Graph

Η βιβλιοθήκη Graph περιέχει την κλάση Graph και VectorNode οι οποίες χρησιμοποιήθηκαν για την αναπαράσταση των KNNG γράφων / ευρετηρίων. Πέρα από τις απαραίτητες λειτουργίες για την δημιουργία και ανακατασκευή των γράφων, περιλαμβάνονται και βοηθητικές λειτουργίες όπως η εκτύπωση ενός γράφου στο τερματικό, η αποθήκευση γράφου σε τοπικό αρχείο και η ανάκτηση γράφου από τοπικό αρχείο.

Input

Η βιβλιοθήκη Input περιλαμβάνει συναρτήσεις για την ανάγνωση των συνόλων δεδομένων διανυσμάτων από αρχεία σε δυαδικό format από την σελίδα http://corpus-texmex.irisa.fr και τη σελίδα https://transactional.blog/sigmod-contest/2024. Επίσης περιλαμβάνεται συνάρτηση η οποία δημιουργεί με brute force το ground truth των συνόλων δεδομένων της 2^{ης} ιστοσελίδας καθώς δεν ήταν διαθέσιμο.

Utilities

Στην βιβλιοθήκη Utilities περιλαμβάνονται συναρτήσεις που εκτελούν σημαντικές λειτουργίες όπως ο υπολογισμός ευκλείδειας απόστασης μεταξύ δύο διανυσμάτων, ο υπολογισμός του recall δοθέντος 2 συνόλων και ο υπολογισμός του medoid ενός συνόλου διανυσμάτων.

Vamana

Στην βιβλιοθήκη Vamana περιλαμβάνονται οι συναρτήσεις που δοθέντος των κατάλληλων παραμέτρων, εκτελούν τους αλγορίθμους: Vamana, FilteredVamana, StitchedVamana, RobustPrune, FilteredRobustPrune, GreedySearch και FilteredGreedySearch.

2.2 Executables

Main Executables

Αρχικά αναπτύχθηκαν τα εξής τρία εκτελέσιμα: `main.cpp`, `main_2.cpp` και `main_3.cpp` τα οποία υλοποιούσαν τα ζητούμενα των ασκήσεων 1, 2 και 3 αντίστοιχα.

Στο `main.cpp` δημιουργείται ένα ευρετήριο-γράφος με τον απλό αλγόριθμο `Vamana` πάνω στο μικρό σύνολο δεδομένων siftsmall του TexMex και εκτελούνται queries πάνω σε αυτό. Εκτυπώνονται επίσης χρόνοι εκτέλεσης των επιμέρους διαδικασιών και recall.

Στο `main_2.cpp` δημιουργείται ευρετήριο-γράφος με τον αλγόριθμο `FilteredVamana` πάνω στο μικρό σύνολο δεδομένων του Sigmod και εκτελούνται queries πάνω σε αυτό. Εκτυπώνονται χρόνοι εκτέλεσης των επιμέρους διαδικασιών, μέσο recall για unfiltered ερωτήματα, μέσο recall για filtered ερωτήματα αλλά και συνολικό μέσο recall.

Στο `main_3.cpp` δημιουργείται ευρετήριο-γράφος με τον αλγόριθμο `StitchedVamana` πάνω στο μικρό σύνολο δεδομένων του Sigmod και εκτελούνται queries πάνω σε αυτό. Εκτυπώνονται χρόνοι εκτέλεσης των επιμέρους διαδικασιών, μέσο recall για unfiltered ερωτήματα, μέσο recall για filtered ερωτήματα αλλά και συνολικό μέσο recall.

Split Executables

Αργότερα, αναπτύξαμε τα παρακάτω εκτελέσιμα στα οποία διαμοιράστηκαν οι διαδικασίες δημιουργίας ευρετηρίου-γράφου και εκτέλεσης των ερωτημάτων για την ευκολότερη εκτέλεση ερωτημάτων, γλυτώνοντας έτσι τον χρόνο που απαιτούσε η δημιουργία τους ευρετηρίου. Ο διαμοιρασμός αυτός επιτεύχθηκε με τις βοηθητικές συναρτήσεις του γράφου για αποθήκευση και ανάκτηση από/σε τοπικό αρχείο.

Vamana	`vamana.sh`	`index_v.cpp`	`query_index_v.cpp`
FilteredVamana	`filtered_vamana.sh`	`index_fv.cpp`	`query_index_fv.cpp`
StitchedVamana	`stitched_vamana.sh`	`index_sv.cpp`	`query_index_sv.cpp`

Αναπτύχθηκαν επίσης και τρία BASH scripts τα οποία κάνουν την εκτέλεση αυτών των εκτελέσιμων ευκολότερη και μέτρηση χρόνου, μνήμης και άλλως μετρικών.

2.3 Tests

Για τον έλεγχο και επαλήθευση της εργασίας αναπτύχθηκαν Unit Tests για κάθε βιβλιοθήκη χρησιμοποιώντας το framework GoogleTest. Τα tests αυτά βρίσκονται μέσα στους φακέλους των βιβλιοθηκών και ελέγχουν απλές και ακραίες περιπτώσεις χρήσης των βιβλιοθηκών αυτών.

Κατά την ανάπτυξη της εργασίας χρησιμοποιήθηκαν επίσης GitHub Actions για την ευκολότερη και ομαλότερη επαλήθευση του κώδικα. Τα αρχεία των workflows βρίσκονται στον φάκελο `.github`

3. Απόδοση – Πειράματα

Σταθερές Τιμές Πειραμάτων

Παράμετροι δημιουργίας ευρετηρίου-γράφου:

Vamana	a=1.2	L=120	R=50	
FilteredVamana	a=1.2	L=120	R=50	
StitchedVamana	a=1.2	L_small=120	R_small=32	R=64

Παράμετροι εκτέλεσης ερωτημάτων:

K=100, L=120

Σύνολα δεδομένων:

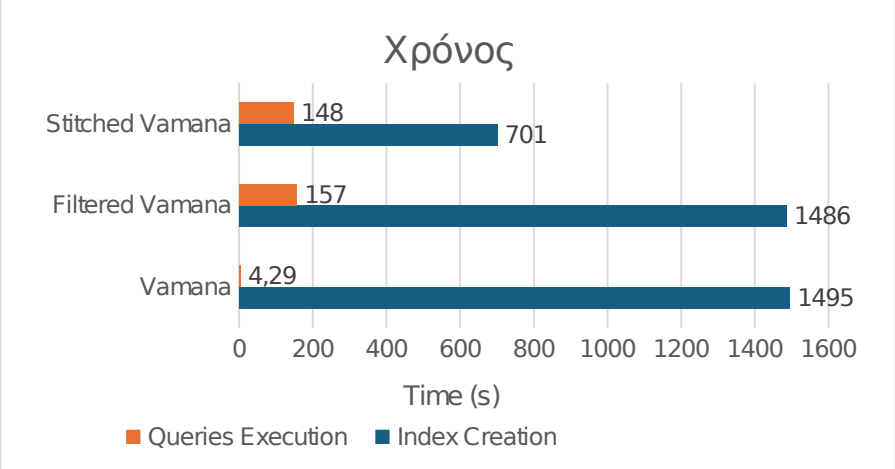
- Vamana: TexMex siftsmall - 10^4 data vectors, 100 queries
- Filtered / Stitched Vamana: Sigmod dummy data - 10^4 data vectors, ~5000 queries

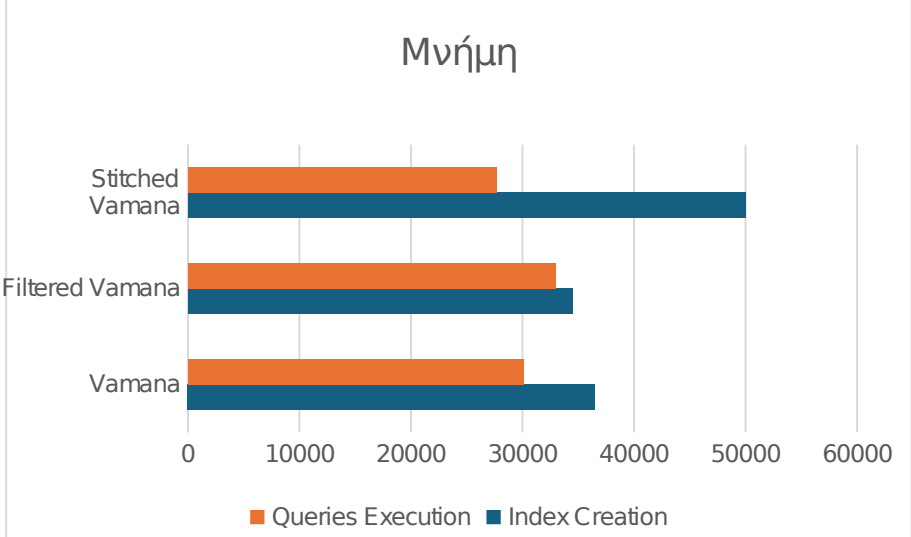
Τεχνικές Βελτιστοποιήσεις

Πριν παρουσιάσουμε την απόδοση των αλγορίθμων και τα πειράματα, θα θέλαμε να αναφέρουμε πως έγιναν διάφορες τεχνικές βελτιστοποιήσεις όσον αφορά τον κώδικα, οι οποίες μείωσαν ελάχιστα το χρόνο εκτέλεσης, αλλά κυρίως μείωσαν την χρήση μνήμης. Τέτοιες βελτιστοποιήσεις ήταν: η αποφυγή χρήσης μαθηματικών συναρτήσεων από τη βιβλιοθήκη `cmath`, Pass-by-Reference μεγάλων δομών όπως γράφων και διανυσμάτων, χρήση `unordered set` και `map` αντί για τα default που είναι sorted και η χρήση των συναρτήσεων `resize` και `reserve` στις αρχικοποιήσεις πινάκων.

3.1 Αρχική Απόδοση

Version #0	Index Creation		Queries Avg. Recall (%)			Queries Execution	
	Time (s)	Max. RSS (kbytes)	Unfiltered	Filtered	Total	Time (s)	Max. RSS (kbytes)
Vamana	1495	36500	0.9993	-	0.9993	4.29	30120
Filtered Vamana	1486	34492	0.230493	0.999519	0.613469	157	32980
Stitched Vamana	701	49976	0.230413	0.981855	0.604632	148	27668





3.2 Βελτιστοποιήσεις

3.2.0 Squared Euclidean Distance

Περιγραφή

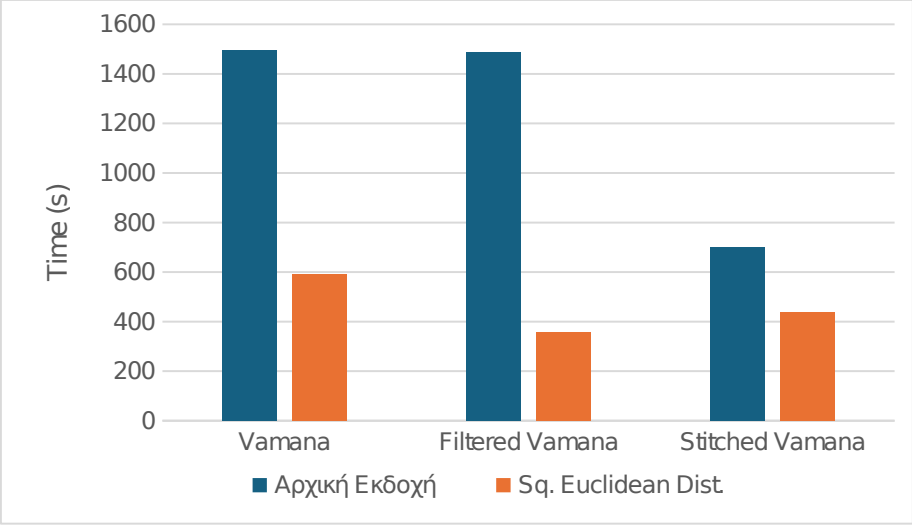
Αντί για τη κλασσική Ευκλείδεια απόσταση επιλέξαμε να δοκιμάσουμε τη Τετραγωνισμένη Ευκλείδεια η οποία αποφεύγει την χρήση της συνάρτησης sqrt() της cmath και δεν αλλάζει τη συμπεριφορά των αλγορίθμων καθώς σημασία έχουν οι συγκρίσεις αποστάσεων μεταξύ των διανυσμάτων.

Αποτελέσματα

Version #1	**Index Creation**		**Queries Avg. Recall (%)**			**Queries Execution**	
Sq. Euclidean Dist.	**Time (s)**	**Max. RSS (kbytes)**	**Unfiltered**	**Filtered**	**Total**	**Time (s)**	**Max. RSS (kbytes)**
Vamana	590	36448	0.9994		0.9994	4.06	24584
Filtered Vamana	359	30704	0.230493	0.999515	0.613467	147	28220
Stitched Vamana	436	47200	0.230441	0.982011	0.604724	136	24964

Παρατηρήσεις

Παρατηρούμε ότι σε σύγκριση με την αρχική εκδοχή υπάρχει σχεδόν υποδιπλασιασμός του χρόνου εκτέλεσης της διαδικασίας δημιουργίας του ευρετηρίου-γράφου για τους αλγορίθμους Vamana και Filtered Vamana. Δεν υπήρξε σημαντική αλλαγή στα recall.



3.2.1 Min Distance

Περιγραφή

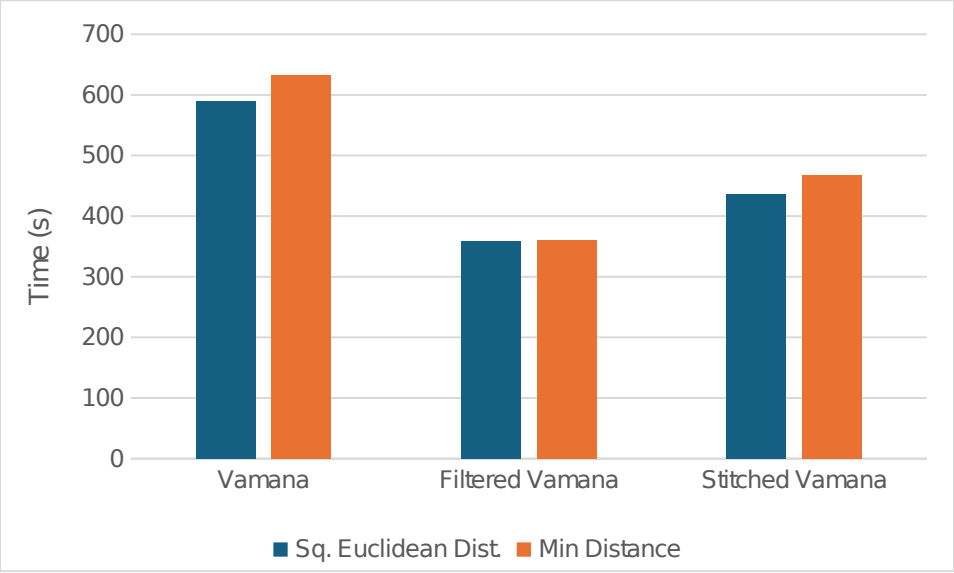
Στον υπολογισμό του medoid εφαρμόσαμε μια βελτιστοποίηση η οποία διακόπτει το άθροισμα των αποστάσεων ενός διανύσματος από τα υπόλοιπα όταν έχει ξεπεράσει το άθροισμα του μέχρι τώρα καλύτερου medoid. Δοκιμάσαμε να βάλουμε παρόμοια βελτιστοποίηση και στον υπολογισμό αποστάσεων σε άλλα σημεία του κώδικα όταν ψάχναμε για ένα διάνυσμα, το κοντινότερο διάνυσμα ενός συνόλου.

Αποτελέσματα

Version #2	**Index Creation**		**Queries Avg. Recall (%)**			**Queries Execution**	
Min. Distance	**Time (s)**	**Max. RSS (kbytes)**	**Unfiltered**	**Filtered**	**Total**	**Time (s)**	**Max. RSS (kbytes)**
Vamana	633	36596	0.9995		0.9995	3.98	24492
Filtered Vamana	360	30832	0.230493	0.999527	0.613473	150	28188
Stitched Vamana	467	47128	0.230445	0.981971	0.604706	137	25040

Παρατηρήσεις

Παρατηρήσαμε ότι υπήρξε μια μικρή αύξηση στον χρόνο για τον απλό Vamana και αμελητέες διαφορές για τους υπόλοιπους οπότε απορρίψαμε αυτή τη βελτιστοποίηση. Δεν υπήρξε σημαντική αλλαγή στα recall.



3.2.2 Πίνακας Αποστάσεων

Περιγραφή

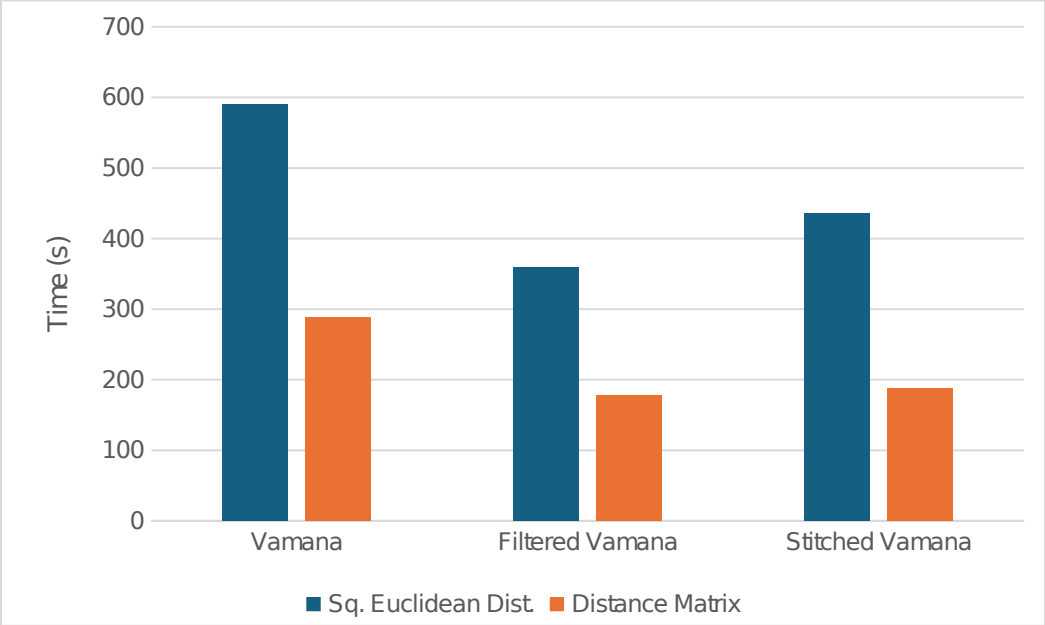
Για τη μείωση του χρόνου της δημιουργίας τους γράφου, προσθέσαμε έναν πίνακα αποστάσεων ο οποίος κρατάει αποστάσεις μεταξύ κόμβων-διανυσμάτων του γράφου για αποφυγή του υπολογισμού τους αργότερα.

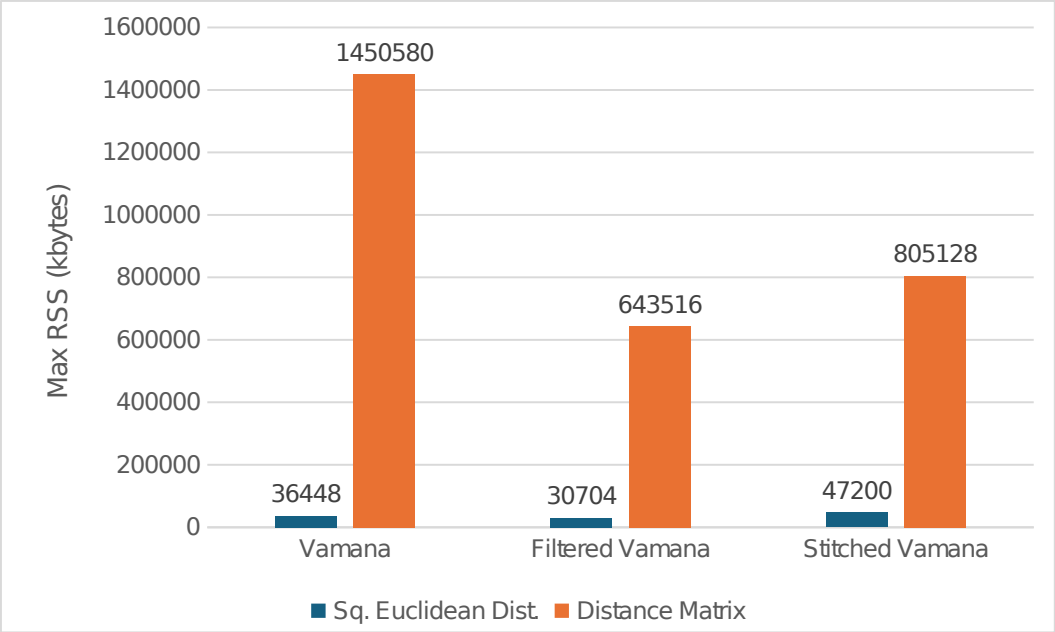
Αποτελέσματα

Version #3	**Index Creation**		**Queries Avg. Recall (%)**			**Queries Execution**	
Distance Matrix	**Time (s)**	**Max. RSS (kbytes)**	**Unfiltered**	**Filtered**	**Total**	**Time (s)**	**Max. RSS (kbytes)**
Vamana	288	1450580	0.9996		0.9996	4	24504
Filtered Vamana	178	643516	0.230493	0.999523	0.613471	149	28212
Stitched Vamana	188	805128	0.230437	0.981951	0.604692	136	24960

Παρατηρήσεις

Παρατηρούμε ότι ο χρόνος και στους 3 αλγορίθμους υποδιπλασιάστηκε αλλά υπήρξε αύξηση x40 σε μνήμη στον απλό Vamana, x20 στον Filtered Vamana και x17 στον Stitched Vamana. Δεν υπήρξε σημαντική αλλαγή στα recall. Συμπεραίνουμε ότι αυτή η βελτιστοποίηση είναι απαγορευτική για μεγάλα σύνολα δεδομένων.




1600000
1400000
1200000
1000000
800000
600000
400000
200000
0
Max RSS (kbytes)
1450580
643516
805128
36448
30704
47200
Vamana
Filtered Vamana
Stitched Vamana
Sq. Euclidean Dist.
Distance Matrix

3.2.3 Αρχικοποίηση Γράφων με τυχαίες ακμές

Περιγραφή

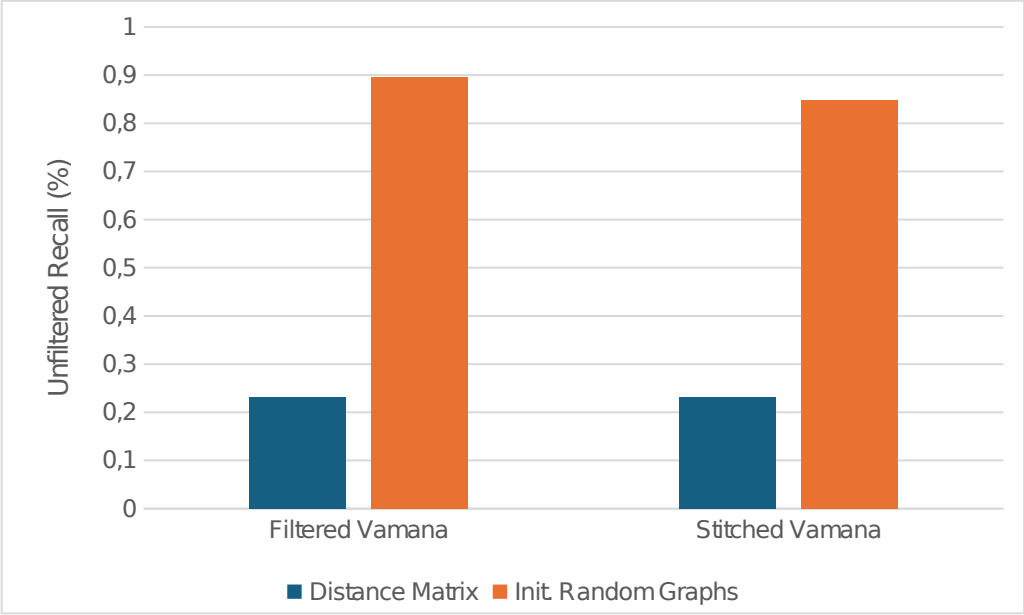
Με την αρχικοποίηση των γράφων με τυχαίες ακμές στον Filtered Vamana και στον Stitched Vamana, περιμένουμε να αυξηθεί το recall των unfiltered queries.

Αποτελέσματα

Version #3.1	**Index Creation**		**Queries Avg. Recall (%)**			**Queries Execution**	
Init. Random Graphs	**Time (s)**	**Max. RSS (kbytes)**	**Unfiltered**	**Filtered**	**Total**	**Time (s)**	**Max. RSS (kbytes)**
Filtered Vamana	218	874140	0.895325	0.999483	0.947193	154	28744
Stitched Vamana	212	1350580	0.848809	0.982003	0.915141	148	26392

Παρατηρήσεις

Ο χρόνος δημιουργίας του γράφου αυξήθηκε ελαφρά, υπήρξε αύξηση στη χρήση της μνήμης ειδικά στον Stitched Vamana όπου αυξήθηκε σχεδόν κατά 60%. Παρατηρούμε επίσης ότι, όπως αναμέναμε, το recall των unfiltered queries αυξήθηκε κατά πολύ ενώ των filtered queries παρέμεινε ίδιο.



3.2.4 Αρχικοποίηση medoid στον Vamana με τυχαία σημεία

Περιγραφή

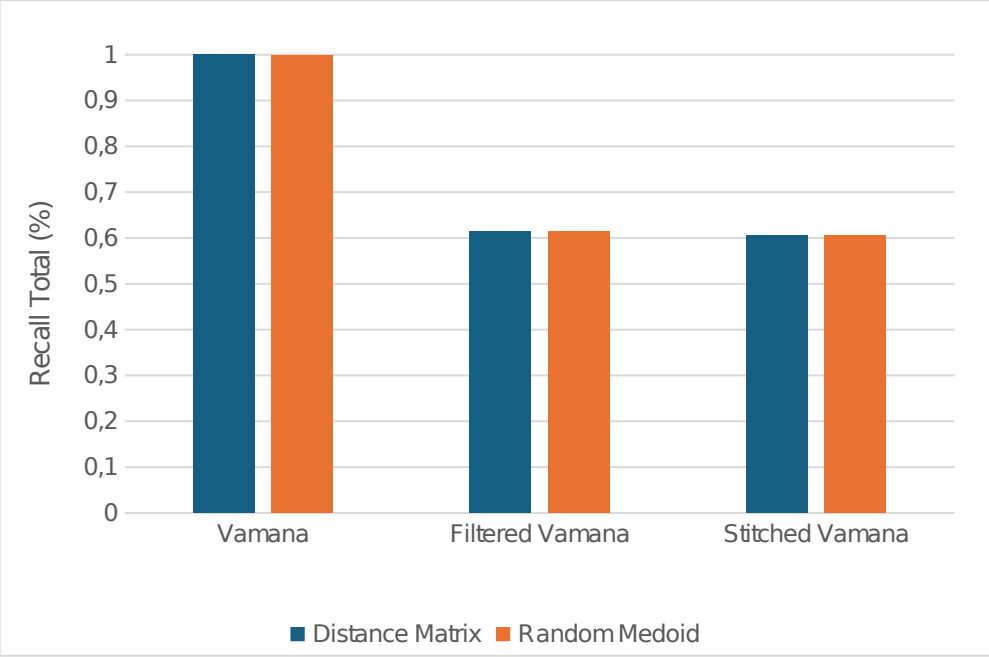
Αντί για τον κανονικό υπολογισμό του medoid για ένα σύνολο διανυσμάτων, επιστρέφουμε ως medoid απλά ένα τυχαίο διάνυσμα.

Αποτελέσματα

Version #3.2	**Index Creation**		**Queries Avg. Recall (%)**			**Queries Execution**	
Random Medoid	**Time (s)**	**Max. RSS (kbytes)**	**Unfiltered**	**Filtered**	**Total**	**Time (s)**	**Max. RSS (kbytes)**
Vamana	201	1453916	0.9993		0.9993	3.8	24536
Filtered Vamana	175	643480	0.230489	0.999527	0.613471	149	28172
Stitched Vamana	171	806456	0.230437	0.982007	0.60472	135	24968

Παρατηρήσεις

Παρατηρούμε πως υπάρχει μείωση 80 δευτερολέπτων στον απλό Vamana και οι χρόνοι των δύο άλλων αλγορίθμων παραμένουν ίδιοι. Δεν υπήρξε αλλαγή στα recall. Συμπεραίνουμε πως και ο υπολογισμός του medoid σε υποσύνολο των διανυσμάτων δεν θα άξιζε καθώς έχουμε τα ίδια αποτελέσματα.



3.3 Παράλληλη Εκτέλεση

3.3.0 OpenMP

Για την εκτέλεση της παραλληλίας χρησιμοποιήσαμε την βιβλιοθήκη της OpenMp.

3.3.1 Υπολογισμός Αποστάσεων και Medoid

Περιγραφή

Επιχειρήσαμε να προσθέσουμε παραλληλία στην εκτέλεση της euclidean distance καθώς και στον υπολογισμό του medoid για τον απλό Vamana.

Αποτελέσματα

Με την προσθήκη παραλληλίας στην κλήση της euclidean distance είδαμε ότι ο χρόνος εκτέλεσης αυξήθηκε σε μεγάλο βαθμό κάτι που κάνει τη χρήση παραλληλίας στο συγκεκριμένο πλαίσιο απαγορευτική.

Με την χρήση παραλληλίας στον υπολογισμό του medoid παρατηρήσαμε ότι ο χρόνος εκτέλεσης της συνάρτησης μειώθηκε αρκετά, καθώς από 90s στη προηγούμενη εκτέλεση πραγματοποιήθηκε σε 36s.

3.3.2 Υπολογισμός ανεξάρτητων Γράφων σε Stitched Vamana

Περιγραφή

Επιχειρήσαμε να προσθέσουμε παραλληλία στην εκτέλεση της κατασκευής των υπογράφων για την συνάρτηση της Stitched Vamana.

Αποτελέσματα

Με την προσθήκη παραλληλίας στην κατασκευή των υπογράφων παρατηρήσαμε ότι ο χρόνος εκτέλεσης μειώθηκε ελάχιστα της τάξης των 5-10s.